10주차 · 11/5(목)

JavaScript 기초 문법 이해

변수, 자료형, 조건문, 반복문 등

한신대학교 AI·SW계열 · 웹프로그래밍

학습사이트 hs-web.dreamitbiz.com

10주차

학습 목표

1. const와 let을 구분해 쓰고 var를 쓰지 않는 이유를 설명할 수 있다.
2. JavaScript의 기본 자료형을 구분하고 형변환의 함정을 피할 수 있다.
3. === 와 == 의 차이를 설명하고 항상 === 를 쓸 수 있다.
4. 조건문과 반복문으로 흐름을 제어할 수 있다.
5. console.log와 개발자도구로 값을 확인하며 문제를 좁혀 갈 수 있다.

10주차

핵심 키워드

const/let · 자료형 · 템플릿 문자열 · 형변환 · === · if/else · for · while · console.log · 디버깅

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

10주차 · 1

JavaScript는 무엇을 하는가

HTML이 뼈대를 세우고 CSS가 꾸몄다면, JavaScript는 페이지를 움직이게 합니다.

JavaScript는 무엇을 하는가

	HTML	CSS	JavaScript
종류	마크업 언어	스타일 언어	프로그래밍 언어
하는 일	무엇이 있는가	어떻게 보이는가	무엇을 하는가
계산·조건·반복	없음	거의 없음	있음

버튼을 누르면 반응하고, 입력한 값을 검사하고, 서버에서 자료를 가져와 화면을 바꾸는 — 그 모든 것이 JavaScript입니다.

JavaScript는 무엇을 하는가

1. 이름에 "Java"가 들어 있지만 Java와는 아무 상관이 없습니다.
2. 1995년에 만들어질 당시 Java가 인기라서 마케팅 목적으로 붙인 이름입니다.
3. 자주 나오는 상식 문제입니다.

CSS와 마찬가지로 세 가지

```
<!-- ① 인라인 — 쓰지 않습니다 -->
<button onclick="alert('hi')">버튼</button>

<!-- ② 내부 script — 연습용 -->
<script>
  console.log('안녕');
</script>

<!-- ③ 외부 파일 — 실무 표준 -->
<script src="app.js"></script>

<!-- script는 </body> 바로 앞에 둡니다.
     head에 두면 HTML이 만들어지기 전에 실행되어
     document.querySelector가 null을 돌려줍니다.
     (또는 head에 두되 defer 속성을 붙입니다) -->
```

10주차 · 2

변수 — 값에 이름 붙이기

const와 let

```
const name = '김한신';    // 다시 대입하지 않을 값
let count = 0;            // 바뀔 값

count = 1;                // ✓ let은 가능
name = '이한신';           // ✗ TypeError: Assignment to constant variable

var old = '옛날 방식';     // ✗ 쓰지 않습니다
```

기본은 const, 바뀌어야 할 때만 let

정리

1. **const를 기본으로**
   씁니다. 값이 안 바뀐다는 것이 코드에 드러나면 읽기 쉬워집니다.
2. **let은 진짜 바뀔 때만**
   카운터, 반복문 변수, 조건에 따라 달라지는 값.
3. **var는 쓰지 않습니다.**
   블록을 무시하고 함수 전체에 퍼지며, 같은 이름으로 다시 선언해도 오류가 안 나서 버그를 만듭니다.

변수 — 값에 이름 붙이기

1. const 로 만든 배열과 객체의 내용은 바꿀 수 있습니다.
2. const arr = [1,2]; arr.push(3); 은 됩니다.
3. const가 막는 것은 "그 이름에 다른 것을 다시 대입하는 것"이지 "안의 내용을 바꾸는 것"이 아닙니다.

규칙과 관례

```
// 규칙 (어기면 오류)
let 1abc;        // ✗ 숫자로 시작 불가
let my-name;     // ✗ 하이픈 불가
let class;       // ✗ 예약어 불가

// 관례 (안 지켜도 되지만 지키세요)
let userName;         // camelCase — 변수 · 함수
const MAX_COUNT = 10; // UPPER_SNAKE — 안 바뀌는 상수
class UserProfile {}  // PascalCase — 클래스

// 이름은 짧게가 아니라 정확하게
let d;                // ✗ 무엇인지 모름
let daysLeft;         // ✓
```

10주차 · 3

자료형

기본형 다섯 + 참조형 둘

```
// 기본형 (primitive)
const s = '문자열';          // string
const n = 42;                // number  ← 정수 · 소수 구분 없음
const b = true;              // boolean
const u = undefined;         // 값이 아직 없음 (선언만 하고 대입 안 함)
const nl = null;             // "비어 있음"을 의도적으로 표시

// 참조형
const arr = [1, 2, 3];       // 배열 (10주차)
const obj = { a: 1 };        // 객체 (10주차)

console.log(typeof s);       // "string"
console.log(typeof n);       // "number"
console.log(typeof nl);      // "object"  ← JavaScript의 유명한 버그입니다
```

따옴표 세 종류

```
const a = '작은따옴표';
const b = "큰따옴표";
const c = `백틱 — 특별합니다`;

const name = '한신';

// ✗ 옛날 방식 — 읽기 어렵다
const msg1 = '안녕하세요, ' + name + '님. 오늘은 ' + 3 + '일입니다.';

// ✓ 템플릿 문자열 — 백틱 안에서 ${ } 로 값을 끼워 넣는다
const msg2 = `안녕하세요, ${name}님. 오늘은 ${1 + 2}일입니다.`;

// 백틱은 여러 줄도 그대로 유지합니다
const html = `
  <div>
    <h1>${name}</h1>
  </div>
`;
```

자주 쓰는 문자열 메서드

```
const s = '  Hello Web  ';

s.length              // 13
s.trim()              // 'Hello Web'  ← 앞뒤 공백 제거 (입력값 검사에 필수)
s.toUpperCase()       // '  HELLO WEB  '
s.includes('Web')     // true
s.replace('Web', '웹') // '  Hello 웹  '
s.split(' ')          // 공백으로 잘라 배열로
s.slice(2, 7)         // 'Hello'  ← 일부 잘라내기

// 문자열은 바뀌지 않습니다(immutable).
// s.toUpperCase()는 새 문자열을 돌려줄 뿐 s 자체는 그대로입니다.
```

여기서 초보자가 가장 많이 당합니다

```
const a = '5';
const b = 3;

console.log(a + b);   // '53'  ← 문자열 이어붙이기! 8이 아닙니다
console.log(a - b);   // 2     ← 빨셈은 숫자로 자동 변환
console.log(a * b);   // 15

// 그래서 반드시 변환해서 씁니다
console.log(Number(a) + b);      // 8
console.log(parseInt('12px'));   // 12   ← 앞에서부터 숫자만
console.log(parseFloat('3.14')); // 3.14

// input.value 는 항상 문자열입니다 — 계산 전에 꼭 변환하세요
const price = Number(document.querySelector('#price').value);
```

숫자 다루기

```
(3.14159).toFixed(2)    // '3.14'  ← 문자열로 돌려줍니다
Math.round(3.6)         // 4
Math.floor(3.9)         // 3  (내림)
Math.ceil(3.1)          // 4  (올림)
Math.max(1, 5, 3)       // 5
Math.random()           // 0 이상 1 미만 난수

// 1~10 무작위 정수
Math.floor(Math.random() * 10) + 1

// 3자리 쉼표
(1234567).toLocaleString('ko-KR')   // '1,234,567'

console.log(0.1 + 0.2);   // 0.30000000000000004  ← 컴퓨터 소수 표현의 한계
```

10주차 · 4

연산자

== 를 쓰지 않는 이유

```
'1' == 1        // true   ← 타입이 달라도 억지로 맞춰 비교
'1' === 1       // false  ← 값과 타입을 함께 비교 ✓

0 == false      // true
0 === false     // false  ✓

null == undefined   // true
null === undefined  // false ✓

// 결론: 항상 === 와 !== 를 씁니다. == 는 예상 못 한 버그의 원인입니다.
```

&& || ! 와 참·거짓 판정

```
const age = 20, hasCard = true;

age >= 19 && hasCard    // 그리고 — 둘 다 참
age < 19 || hasCard     // 또는 — 하나라도 참
!hasCard                // 부정

// 거짓으로 취급되는 값 (falsy) — 이 여섯 개만 외우면 됩니다
false, 0, '', null, undefined, NaN

// 나머지는 전부 참 (truthy) — 빈 배열 []과 빈 객체 {}도 참입니다!
if ([]) console.log('빈 배열도 참입니다');   // 실행됩니다
```

실무에서 자주 쓰는 두 가지

```
// 기본값 주기 (||)
const nickname = input || '익명';   // input이 비었으면 '익명'

// null · undefined일 때만 기본값 (??)  ← 0과 ''은 그대로 두고 싶을 때
const count = value ?? 0;

// 안전하게 접근하기 (?.)
const city = user?.address?.city;   // user나 address가 없어도 오류가 안 남
```

짧은 분기

```
const result = score >= 60 ? '합격' : '불합격';

// if로 쓰면
let result2;
if (score >= 60) result2 = '합격';
else result2 = '불합격';

// 중첩은 하지 마세요 — 읽기 어려워집니다
const g = s >= 90 ? 'A' : s >= 80 ? 'B' : s >= 70 ? 'C' : 'F';  // ✗ 이건 if로
```

10주차 · 5

조건문

if / else if / else

```
const score = 85;

if (score >= 90) {
  console.log('A');
} else if (score >= 80) {
  console.log('B');       // ← 여기가 실행됩니다
} else if (score >= 70) {
  console.log('C');
} else {
  console.log('F');
}

// 위에서부터 순서대로 검사하고, 처음 맞는 곳에서 멈춥니다.
// 조건 순서를 거꾸로(70부터) 쓰면 전부 C가 됩니다.
```

switch — 값이 딱 떨어질 때

```
const day = 4;

switch (day) {
  case 1: console.log('월'); break;   // break를 빠뜨리면 아래로 계속 흘러갑니다
  case 4: console.log('목'); break;
  default: console.log('그 외');
}
```

10주차 · 6

반복문

for — 횟수를 아는 반복

```
for (let i = 0; i < 5; i++) {
  console.log(i);      // 0, 1, 2, 3, 4
}
//    ①         ②       ③
// ① 시작: 한 번만 실행
// ② 조건: 매번 검사, 참이면 계속
// ③ 증가: 매 반복 끝에 실행

// i는 0부터 시작하고 < n 까지가 관례입니다.
// i <= n 으로 쓰면 한 번 더 돌아 배열 범위를 넘습니다.
```

for...of — 배열을 훑을 때 (가장 많이 씁니다)

```
const fruits = ['사과', '배', '감'];

for (const fruit of fruits) {
  console.log(fruit);      // 사과, 배, 감
}

// 순서(인덱스)도 필요하면
for (const [i, fruit] of fruits.entries()) {
  console.log(`${i + 1}번: ${fruit}`);
}
```

while — 조건이 참인 동안

```
let n = 1;
while (n <= 5) {
  console.log(n);
  n++;                // 이걸 빠뜨리면 무한 루프! 브라우저가 멈춥니다
}

// break: 반복을 완전히 빠져나감
// continue: 이번 회차만 건너뛰고 다음으로
for (let i = 0; i < 10; i++) {
  if (i === 3) continue;   // 3만 건너뜀
  if (i === 6) break;      // 6에서 멈춤
  console.log(i);          // 0 1 2 4 5
}
```

반복문

1. 무한 루프에 빠지면 브라우저 탭이 얼어붙습니다.
2. 그럴 때는 탭을 닫고 코드를 고친 뒤 다시 여세요.
3. while 을 쓸 때는 탈출 조건이 언젠가 참이 되는지 먼저 확인하는 습관을 들이세요.

10주차 · 7

콘솔 — 이 학기 내내 쓸 도구

값을 확인하는 여러 방법

```
console.log('값:', value);        // 가장 많이 씀
console.log({ name, age });       // 변수 이름과 값을 함께 (아주 유용!)
console.table(arrayOfObjects);    // 배열을 표로
console.warn('주의');
console.error('오류');
console.time('작업'); /* ... */ console.timeEnd('작업');   // 걸린 시간
```

"왜 안 되지?" 할 때의 순서

정리

1. **F12 → Console 탭을 먼저 엽니다.**
   빨간 글씨가 있으면 거기에 파일명과 줄 번호가 적혀 있습니다.
2. **의심되는 곳 앞뒤에 console.log 를 찍습니다.**
   어디까지 실행됐는지, 값이 무엇인지 확인합니다.
3. **값이 undefined 나 null 이면**
   그 앞 단계에서 이미 실패한 것입니다. 더 위로 올라가며 찾습니다.
4. **혼자 30분 이상 막히면 물어보세요.**
   그것은 인내가 아니라 낭비입니다.

콘솔 — 이 학기 내내 쓸 도구

1. 완성한 코드에서는 console.log 를 지우세요.

2. 남겨 두면 사용자의 콘솔에 그대로 보이고, 실수로 개인정보를 찍고 있을 수도 있습니다.

10주차

실습

이 시간의 실습 4개 — 학습사이트의 코드 실습장에서 직접 치고 실행합니다

실습 1 · 기본

실습

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

> 이름과 학년을 변수에 담고, 템플릿 문자열로 "OOO님은 O학년입니다"를 콘솔에 출력하세요. 두 변수는 바뀌지 않으므로 적절한 키워드를 고르세요.

힌트

바뀌지 않는 값은 const입니다. 템플릿 문자열은 백틱(`)과 ${ } 입니다.

실습 1 모범답안

모범답안

```
const name = '김한신';
const year = 1;

console.log(`${name}님은 ${year}학년입니다`);
```

실습 2 · 기본

실습

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

> 점수를 담은 변수를 만들고, 90 이상 A, 80 이상 B, 70 이상 C, 나머지는 F를 출력하세요. 등급을 변수에 담았다가 마지막에 한 번만 출력하세요.

힌트

조건은 큰 값부터 검사해야 합니다. 작은 값부터 쓰면 전부 첫 조건에 걸립니다.

실습 2 모범답안

모범답안

```
const score = 85;
let grade;

if (score >= 90) {
  grade = 'A';
} else if (score >= 80) {
  grade = 'B';
} else if (score >= 70) {
  grade = 'C';
} else {
  grade = 'F';
}

console.log(`${score}점은 ${grade}학점입니다`);
```

실습 3 · 응용

실습

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

> 구구단 3단을 콘솔에 출력하세요. "3 x 1 = 3" 형태로 9줄이 나와야 합니다.

힌트

for 문에서 i를 1부터 9까지 돌리고, 템플릿 문자열로 만듭니다.

실습 3 모범답안

모범답안

```
const dan = 3;

for (let i = 1; i <= 9; i++) {
  console.log(`${dan} x ${i} = ${dan * i}`);
}
```

실습 4 · 심화

실습

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

> 아래 코드는 8이 나와야 하는데 "53"이 나옵니다. 원인을 찾아 고치세요. 그리고 왜 그런지 주석으로 적으세요.

힌트

입력값은 언제나 문자열입니다. + 는 문자열끼리는 이어붙이기가 됩니다.

실습 4 모범답안

모범답안

```
const a = '5';
const b = 3;

// a는 문자열이라 + 가 "이어붙이기"로 동작해 '53'이 됩니다.
// 계산하려면 숫자로 바꿔야 합니다.
console.log(Number(a) + b);   // 8
```

10주차 마무리

10주차 마무리

JavaScript 기초 문법 이해 — 이 시간에 다룬 것을 학습사이트에서 다시 확인합니다

- ✓ 학습사이트 주차 페이지의 절마다 「이해했어요 · 실습했어요 · 어려워요」 표시
- ✓ 실습 4개를 실습장에서 「확인」 까지 — 「실습했어요」 체크
- ✓ 퀴즈 8 「퀴즈 8 — JavaScript 기초 문법」 응시 (성적 미반영 · 학습 확인용)
- ✓ 막히면 「질문있어요」 또는 패들릿에 — 30분 넘게 혼자 막혀 있지 않기

다음 시간

11주차 · JavaScript 코어 객체 활용